Phase 7: 训练基础设施实现指导

1. AdamW 优化器

1.1 继承 torch.optim.Optimizer 的模板

```
class AdamW(torch.optim.Optimizer):
    def __init__(self, params, lr, betas=(0.9, 0.999), eps=1e-8,
weight_decay=0.01):
        # defaults 是一个 dict，存储所有超参数
        defaults = dict(lr=lr, betas=betas, eps=eps,
weight_decay=weight_decay)
        super().__init__(params, defaults)

    def step(self, closure=None):
        # 遍历 param_groups（通常只有一组）
        for group in self.param_groups:
            lr = group['lr']
            beta1, beta2 = group['betas']
            eps = group['eps']
            wd = group['weight_decay']

            for p in group['params']:
                if p.grad is None:
                    continue

                grad = p.grad.data
                state = self.state[p]

                # 懒初始化：第一次访问时创建
                if len(state) == 0:
                    state['step'] = 0
                    state['m'] = torch.zeros_like(p.data)  # 一阶矩
                    state['v'] = torch.zeros_like(p.data)  # 二阶矩

                state['step'] += 1
                t = state['step']
                m, v = state['m'], state['v']

                # ... 更新逻辑 ...
```

1.2 更新公式的精确顺序

AdamW 的更新步骤（注意 weight decay 是解耦的）：

```
t += 1
m = β₁ * m + (1 - β₁) * g
v = β₂ * v + (1 - β₂) * g²
```

```
# 偏差校正
m̂ = m / (1 - β₁^t)
v̂ = v / (1 - β₂^t)

# 参数更新（weight decay 解耦）
θ = θ - lr * (m̂ / (√v̂ + ε) + wd * θ)
```

1.3 原地操作的重要性

```
# 必须使用原地操作更新 m, v, p
# 因为 state 中保存的是引用
m.mul_(beta1).add_(grad, alpha=1 - beta1)     # m = β₁*m + (1-β₁)*g
v.mul_(beta2).addcmul_(grad, grad, value=1 - beta2)  # v = β₂*v + (1-β₂)*g²

# 或更直白的写法:
m.mul_(beta1).add_(grad * (1 - beta1))
v.mul_(beta2).add_(grad * grad * (1 - beta2))

# 参数更新也需要原地
p.data.add_(update, alpha=-lr)
```

1.4 PyTorch AdamW 可能的差异

测试先对比 PyTorch 的 AdamW 结果。如果不匹配，则对比参考快照。

差异来源可能在于 weight decay 的具体应用位置：

- 先 decay 再 Adam 更新
- 或 Adam 更新和 decay 合并

两种都是"正确的" AdamW，只是浮点行为略有不同。

1.5 power 运算

```
# β₁^t 的计算
bias_correction1 = 1 - beta1 ** t
bias_correction2 = 1 - beta2 ** t

# 或者更高效地递推（但直接用 ** 对于本作业足够）
```

---

2. Cosine LR Schedule

2.1 精确公式

```
if t < T_w:
    lr = (t / T_w) * α_max
elif t <= T_c:
    lr = α_min + 0.5 * (α_max - α_min) * (1 + cos(π * (t - T_w) / (T_c -
T_w)))
else:
    lr = α_min
```

2.2 注意测试中的边界条件

看测试数据：

- `t=0` → lr = 0（warmup 从 0 开始）
- `t=T_w=7` → lr = α_max = 1.0（warmup 结束时刚好达到最大值）
- `t=T_c=21` → lr = α_min = 0.1（cosine 结束时刚好达到最小值）
- `t > T_c` → lr = α_min = 0.1

2.3 Python math 模块

```
import math
math.cos(math.pi * ratio)  # ratio ∈ [0, 1]
```

2.4 在训练循环中使用

每步训练前更新学习率：

```
for step in range(total_steps):
    lr = get_lr_cosine_schedule(step, max_lr, min_lr, warmup_iters,
cosine_cycle_iters)
    for group in optimizer.param_groups:
        group['lr'] = lr

    # ... 正常训练步骤 ...
```

---

3. Cross-Entropy Loss

3.1 数值稳定的实现思路

```
输入:
  logits: (N, V)   — N 个样本, V 个类别
  targets: (N,)    — 每个样本的正确类别索引

计算:
  1. max_logit = logits.max(dim=-1, keepdim=True)    (N, 1)
```

```
  2. shifted = logits - max_logit                      (N, V)
  3. log_sum_exp = log(sum(exp(shifted), dim=-1))      (N,)
  4. target_logit = shifted[range(N), targets]         (N,)  ← 用高级索引
  5. loss_per_sample = -target_logit + log_sum_exp     (N,)
  6. mean_loss = loss_per_sample.mean()                ()
```

3.2 PyTorch 高级索引取目标值

```
# logits: (N, V)
# targets: (N,)
# 取每行中 target 列的值:
target_logits = logits[torch.arange(N), targets]   # (N,)

# 或
target_logits = logits.gather(dim=-1,
index=targets.unsqueeze(-1)).squeeze(-1)
```

---

4. Gradient Clipping

4.1 实现步骤

```
1. total_norm_sq = 0
   for p in parameters:
       if p.grad is not None:
           total_norm_sq += (p.grad ** 2).sum()
   total_norm = sqrt(total_norm_sq)

2. if total_norm > max_norm:
       clip_coef = max_norm / (total_norm + 1e-6)
       for p in parameters:
           if p.grad is not None:
               p.grad.mul_(clip_coef)       # 原地缩放
```

4.2 注意事项

- 参数可能设了 `requires_grad_(False)`，此时 `p.grad` 为 None，要跳过
- 测试与 `torch.nn.utils.clip_grad_norm_` 对比
- eps = 1e-6（作业要求）

---

5. Data Loading (get_batch)

5.1 核心逻辑

```
dataset: numpy array, 形状 (N,)
batch_size: B
context_length: m
device: 'cpu' 或 'cuda:0' 等

1. 随机采样 B 个起始位置:
   starts = np.random.randint(0, N - m, size=B)
   # 注意范围: [0, N - m - 1]，确保能取到 m+1 个 token（inputs + 1 个 label）

   等等——仔细看测试:
   test 验证 max(starting_indices) == len(dataset) - context_length - 1
   即 starts ∈ [0, N - m - 1] (含两端)
   → np.random.randint(0, N - m)  ← 上界不含，所以 [0, N-m-1]

2. 对每个 start:
   inputs[i] = dataset[start : start + m]
   labels[i] = dataset[start + 1 : start + m + 1]

3. 转为 torch.LongTensor 并移到 device
```

5.2 PyTorch 操作

```
# numpy → torch
x = torch.from_numpy(x_np).long()

# 或直接用 torch
x = torch.tensor(x_np, dtype=torch.long)

# 移到 device
x = x.to(device)

# 注意: 如果 device='cuda:99'（不存在），会抛出 RuntimeError
# 测试会检查这个错误
```

5.3 大数据集的内存映射

```
# 训练时用 memmap 加载
dataset = np.memmap(path, dtype=np.uint16, mode='r')

# 或者如果保存为 .npy:
dataset = np.load(path, mmap_mode='r')

# memmap 不会把整个文件读入内存
# 只在实际访问某个位置时才从磁盘读取
# 索引操作（如 dataset[start:start+m]）会触发按需读取
```

---

6. Checkpointing

6.1 保存

```
def save_checkpoint(model, optimizer, iteration, out):
    checkpoint = {
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'iteration': iteration,
    }
    torch.save(checkpoint, out)
```

6.2 加载

```
def load_checkpoint(src, model, optimizer):
    checkpoint = torch.load(src)
    model.load_state_dict(checkpoint['model_state_dict'])
    optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
    return checkpoint['iteration']
```

6.3 torch.save / torch.load 的接口

```
# 支持路径字符串
torch.save(obj, '/path/to/file.pt')
torch.load('/path/to/file.pt')

# 也支持 file-like object (测试中可能传 BytesIO 或 file handle)
import io
buffer = io.BytesIO()
torch.save(obj, buffer)
buffer.seek(0)
loaded = torch.load(buffer)
```

6.4 测试注意事项

- 测试用你的 AdamW 创建优化器，所以必须先通过 AdamW 测试
- 加载后模型参数必须与保存前完全一致
- 优化器状态（m, v, step）也必须完全一致
- `torch.load` 可能需要 `weights_only=False`（取决于 PyTorch 版本）

---

7. 训练循环（训练脚本）

7.1 基本结构

```python
# 伪代码框架
for step in range(total_steps):
    # 1. 获取学习率
    lr = get_lr(step, ...)
    set_lr(optimizer, lr)

    # 2. 采样 batch
    inputs, targets = get_batch(train_data, batch_size, ctx_len, device)

    # 3. Forward
    logits = model(inputs)
    loss = cross_entropy(logits.view(-1, vocab_size), targets.view(-1))

    # 4. Backward
    optimizer.zero_grad()
    loss.backward()

    # 5. Gradient clipping
    gradient_clipping(model.parameters(), max_norm)

    # 6. Update
    optimizer.step()

    # 7. Logging
    if step % log_interval == 0:
        print(f"step {step}: train_loss = {loss.item():.4f}")

    # 8. Validation
    if step % eval_interval == 0:
        val_loss = evaluate(model, val_data, ...)
        print(f"step {step}: val_loss = {val_loss:.4f}")

    # 9. Checkpointing
    if step % save_interval == 0:
        save_checkpoint(model, optimizer, step, path)
```

7.2 评估函数

```python
@torch.no_grad()
def evaluate(model, val_data, num_batches, batch_size, ctx_len, device):
    model.eval()
    losses = []
    for _ in range(num_batches):
        inputs, targets = get_batch(val_data, batch_size, ctx_len, device)
        logits = model(inputs)
        loss = cross_entropy(logits.view(-1, vocab_size), targets.view(-1))
        losses.append(loss.item())
    model.train()
    return sum(losses) / len(losses)
```

7.3 wandb 日志（可选但推荐）

```
import wandb

wandb.init(project="cs336-a1", config={...})

# 在训练循环中:
wandb.log({"train_loss": loss.item(), "lr": lr, "step": step})
wandb.log({"val_loss": val_loss, "step": step})
```

7.4 命令行参数

```
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("--d_model", type=int, default=512)
parser.add_argument("--num_layers", type=int, default=4)
parser.add_argument("--lr", type=float, default=1e-3)
# ... 更多参数
args = parser.parse_args()
```